KV CACHE & ATTENTION

Expanded class notes with step-by-step explanations of MHA, MQA, autoregressive caching, and encoder/decoder attention

Purpose of this revision
These notes retain the original class topics and add a clearer learning path for the concepts that needed more explanation. The numerical examples are deliberately small and illustrative; real models learn their projection weights during training.

Learning path

1. Transformer foundations → 2. Q, K, V and MHA → 3. Sequence dimensions vs head dimensions → 4. MQA and shared K/V → 5. Token-to-token attention → 6. Causal masking → 7. KV cache and decoding → 8. Encoder, decoder and cross-attention → 9. Memory calculations and revision.

Use this as a handwriting-friendly study document. Copy the diagrams, shapes, and key equations rather than every explanatory sentence.

1. Transformer foundations

Start with the data flow before learning attention optimisations.

Tokenisation: text is converted into token IDs. An embedding layer maps each token ID to a vector. Positional information is also added so the model can represent token order.

Example sentence: **“The cat chased the mouse”**. Assume a simplified tokenizer produces five tokens.

Token position	1	2	3	4	5
Token	The	cat	chased	the	mouse

If d_model = 512 and the sequence has 5 tokens, the input matrix has shape:

$$X \in R^{5 \times 512} \text{ (sequence length } \times \text{ model dimension)}$$

Self-attention: each token builds a representation using information from other permitted tokens in the same sequence. The permitted positions depend on the attention mask.

> **Important distinction**
> The sequence dimension counts token positions. The feature/model dimension describes the vector width of each token. Attention heads split the feature dimension, not the sequence.

Autoregressive generation predicts one next token at a time. The generated token is fed back as part of the input for the next decoding step.

2. Q, K, V and Multi-Head Attention (MHA)

Understand the baseline before comparing it with MQA.

Each input token is transformed into Query (Q), Key (K), and Value (V) vectors using learned projection matrices:

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

Who chooses the projection rules? The model designer chooses the architecture and matrix shapes. The numerical values inside WQ, WK and WV are learned during training using backpropagation and an optimiser. They are not manually assigned as “take the first two values” or “take the last two values.”

Symbol	Intuition	Role in attention
Q — Query	What am I looking for?	Compared with keys to produce attention scores.
K — Key	What can I be matched against?	Provides the representations against which queries are compared.
V — Value	What information do I contribute?	Combined using the attention weights.

Scaled dot-product attention:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(QK^T / \sqrt{d_k}\right) V$$

For each query position, QK■ produces scores against the key positions. Softmax converts the scores into weights. Those weights form a weighted sum of the value vectors.

MHA example: d_model = 512, number of heads = 8. Therefore head_dim = 512 / 8 = 64.

MHA component	Output dimension	After splitting
Q projection	512	8 query heads × 64
K projection	512	8 key heads × 64
V projection	512	8 value heads × 64

MHA mental model
Every query head has its own corresponding K and V head. Each head attends over the token positions, produces an output, and the head outputs are concatenated and passed through an output projection.

3. Sequence dimension vs head dimension

This distinction prevents a common misunderstanding.

Assume a tiny example: sequence length = 5, d_model = 4, number of heads = 2, head_dim = 2.

The input matrix X has shape 5 × 4. After Q, K and V projections in MHA, each matrix still has shape 5 × 4. The feature width 4 is split into two groups of 2 for each token.

Token	Head 1 features	Head 2 features
The	Q1/K1/V1: 2 values	Q2/K2/V2: 2 values
cat	Q1/K1/V1: 2 values	Q2/K2/V2: 2 values
chased	Q1/K1/V1: 2 values	Q2/K2/V2: 2 values
the	Q1/K1/V1: 2 values	Q2/K2/V2: 2 values
mouse	Q1/K1/V1: 2 values	Q2/K2/V2: 2 values

Each head still has representations for **all five token positions**. We do not assign “The” to Head 1 and “cat” to Head 2. Every head processes the sequence; the heads differ in their learned feature projections.

> **Remember**
> Split the feature dimension of every token into heads. Do not split the sequence into separate groups of tokens.

Useful shape notation

```
Before splitting: [sequence_length, d_model] = [5, 4]
After splitting: [num_heads, sequence_length, head_dim] = [2, 5, 2] (ignoring batch dimension).
```

4. Multi-Query Attention (MQA)

What is shared, and how does the architecture create sharing?

MQA keeps multiple query heads but uses only one key head and one value head. The K and V projections are designed to output one head-sized vector per token, rather than one vector per query head.

Tiny shape example: d_model = 4, query heads = 2, head_dim = 2.

Projection	MHA matrix shape	MQA matrix shape	Result
WQ	4 × 4	4 × 4	2 query heads
WK	4 × 4	4 × 2	1 key head
WV	4 × 4	4 × 2	1 value head

Use row-vector convention: X [5 × 4] multiplied by WK [4 × 2] gives K [5 × 2]. For each token there is one 2-dimensional K vector. There is no second K head to split out.

Likewise, X [5 × 4] × WV [4 × 2] gives V [5 × 2]. Every query head reads this same K/V representation for each token.

What “shared” means
MQA does not first create several K/V heads and then merge them. It creates one K head and one V head per token. All query heads are connected to that same K/V head during attention.

Original 512-dimensional example

Component	MHA: 8 heads	MQA: 8 query heads
Q output	512 = 8 × 64	512 = 8 × 64
K output	512 = 8 × 64	64 = 1 × 64
V output	512 = 8 × 64	64 = 1 × 64
KV heads	8	1

The original MQA paper describes the different query heads sharing a single set of keys and values. Its implementation removes the head dimension from the K/V projection operations. The query heads remain.

Training vs inference: projection weights are learned during training. During inference, heads read the shared K/V vectors; they do not independently modify the cached vectors.

5. How one token attends to other tokens

The sequence dimension stays intact; attention happens across token positions.

Consider “The cat chased the mouse”. Focus on the token “mouse”. Its query is compared with the keys of all positions it is allowed to attend to.

For the final token in a causal sequence, the allowed positions are The, cat, chased, the, mouse.

```
Scores = Q_mouse × K■
Weights = softmax(scores / √d_k)
Output_mouse = weights × V
```

Key position	Illustrative attention weight	Interpretation
The	0.05	Small contribution
cat	0.25	Some contribution
chased	0.45	Largest contribution in this illustration
the	0.05	Small contribution
mouse	0.20	Self-position contribution

These weights are illustrative only; actual weights depend on the trained model and input. The output is a weighted combination of the value vectors at the allowed positions.

Why MHA can be richer
Different MHA heads have distinct learned K/V projections, so they can form different representations and attention patterns. MQA keeps different queries and attention calculations, but all query heads use the same K/V representation. MQA can therefore have less attention diversity.

Important: MQA does not lose access to earlier tokens. It still attends across the same sequence positions as MHA. What is reduced is the number of distinct K/V representations per position.

6. Causal masking and bidirectional attention

Why adding a future token does not change previous decoder states.

A causal mask blocks a token from attending to future positions. In the matrix below, ✓ means allowed and — means blocked.

Attending token	The	cat	chased	the	mouse
The	✓	—	—	—	—
cat	✓	✓	—	—	—
chased	✓	✓	✓	—	—
the	✓	✓	✓	✓	—
mouse	✓	✓	✓	✓	✓

Now append “and”. The old five tokens still cannot see future positions, so their allowed attention relationships do not change. The new token “and” can attend to all six positions up to itself.

Attending token	The	cat	chased	the	mouse	and
The	✓	—	—	—	—	—
cat	✓	✓	—	—	—	—
chased	✓	✓	✓	—	—	—
the	✓	✓	✓	✓	—	—
mouse	✓	✓	✓	✓	✓	—
and	✓	✓	✓	✓	✓	✓

Why old K/V can be reused in causal decoding
In a standard causal decoder, earlier positions cannot attend to a newly appended future token. Their layer representations, and therefore their K/V for that layer, remain unchanged. The new token gets its own Q/K/V.

Terminology: “causal” attention is often called masked self-attention. “Causal” is the correct spelling (not casual).

7. KV cache during autoregressive generation

What is recomputed, reused, and appended?

Without a KV cache: at each generation step, the model processes the growing sequence and recomputes K/V for earlier positions. This repeats work.

With a KV cache: during prefill, process the prompt and store K/V. During each decode step, compute Q/K/V for the newly processed token, reuse old cached K/V, and append the new K/V.

Worked timeline

Stage	Sequence processed	Cache action
Prefill	The cat chased the mouse	Compute and store K/V for all 5 prompt tokens.
Decode 1	and	Compute K/V for “and”; append. Cache length becomes 6.
Decode 2	then	Compute K/V for “then”; append. Cache length becomes 7.
Decode 3	ran	Compute K/V for “ran”; append. Cache length becomes 8.

MHA vs MQA for each new token

Operation	MHA (2 heads)	MQA (2 query heads)
New Q	2 query heads	2 query heads
New K/V	2 K heads + 2 V heads	1 shared K head + 1 shared V head
Old cache	Reuse all previous heads	Reuse all previous shared K/V

Precise generation detail
A model predicts the next token from the current context. Once that token is selected, it is fed into the next forward pass. That next pass computes the selected token’s Q/K/V and updates the cache so the model can predict the token after it.

Each Transformer layer maintains its own K/V cache. Q is generally not cached because it is needed for the current attention calculation, while old K/V are needed by future queries.

8. Encoder vs decoder vs cross-attention

Encoder models also use self-attention; the attention direction is different.

Encoder self-attention: typically bidirectional. Every input token can attend to every other unmasked input token. The encoder normally processes the complete input sequence together and produces contextual representations.

Decoder self-attention: causal. A decoder token can attend only to earlier positions and itself. This supports incremental generation and ordinary decoder KV caching.

Attention type	Can attend to future input positions?	Typical cache behaviour
Encoder bidirectional self-attention	Yes	Usually process the full input; appending a token can change earlier contextual states.
Decoder causal self-attention	No	Reuse previous K/V; compute and append K/V for the new token.
Decoder cross-attention	Decoder queries attend to encoder outputs	Encoder-derived K/V can be computed once and reused while decoding.

Why encoder K/V usually cannot be incrementally appended like decoder K/V: when a new token is added to a bidirectional encoder input, previous tokens are allowed to attend to it. Their contextual representations can therefore change. A standard decoder-style append-only cache would not reproduce a fresh full encoder pass.

Encoder-decoder Transformer flow

```
Source sentence → Encoder (bidirectional self-attention) → Encoder outputs → Decoder (causal self-attention + cross-attention) → generated target tokens
```

During decoding, two reusable contexts may exist: (1) decoder self-attention K/V from previously processed target tokens; (2) cross-attention K/V projected from the fixed encoder outputs. The decoder's current query changes at each step.

> **Do not mix these up**
> Encoder self-attention is still self-attention. Cross-attention is different: decoder queries attend to encoder outputs. The encoder output is fixed during ordinary target-side autoregressive decoding.

Qualification: specialised encoder systems may support incremental or prefix caching under specific constraints. The standard bidirectional encoder generally recomputes contextual representations if the input sequence changes.

9. KV-cache memory and inference performance

The main reason MQA is useful for decoding.

KV cache stores Key and Value tensors for processed tokens, for every layer. It grows with sequence length, batch size, number of KV heads, head dimension, number of layers, and bytes per stored value.

```
KV bytes per token = 2 × num_KV_heads × head_dim × num_layers × bytes_per_value
```

```
Total KV bytes = KV bytes per token × sequence_length × batch_size
```

Example from the class notes: 8 KV heads × 64 head dimension × 32 layers × 2 (K and V) × 2 bytes (FP16) = 65,536 bytes = 64 KiB per token. For 10,000 tokens, this is approximately 640 MiB (using decimal/rounded convention in class notes).

MHA vs MQA: reducing KV heads from 8 to 1 reduces the K/V cache by a factor of 8, all else equal. With 32 query heads and one KV head, the reduction factor is 32.

Memory wall: GPU arithmetic throughput can be much higher than memory bandwidth. During incremental decoding, repeatedly moving large K/V tensors can limit speed. MQA reduces K/V data that must be stored and moved.

Prefill and decode

Phase	What happens	Performance measure
Prefill	Process the prompt in parallel and build the KV cache.	TTFT — time to first token.
Decode	Generate one token at a time, reusing the cache.	TPOT — time per output token; TPS — tokens per second.

10. Final revision sheet

Use this page to test whether the full mental model is connected.

Concept	Remember
Projection matrices	Architecture sets shapes; training learns numerical weights.
MHA	Each query head has its own K and V head.
MQA	Many query heads share one K head and one V head.
Head splitting	Split feature dimensions for every token; do not split token positions.
Self-attention	A token's Q is compared with permitted tokens' K; weights combine their V.
Causal mask	Prevents a position from attending to future positions.
KV cache	Stores previous K/V per layer; old entries are reused during causal decoding.
New generated token	Its Q/K/V are computed when it is processed in the next forward pass; K/V are appended.
Encoder	Typically bidirectional self-attention; a changed input can alter earlier contextual states.
Cross-attention	Decoder queries attend to fixed encoder outputs; encoder-derived K/V can be reused.

Five self-check questions

1. With d_model = 512 and 8 heads, what is head_dim?
2. In MQA, how many K/V heads exist if there are 8 query heads?
3. Does MQA prevent a query head from attending to earlier token positions?
4. Why can old decoder K/V be reused after a new token is appended?
5. Why is ordinary bidirectional encoder attention different from causal decoder attention for incremental caching?

One-sentence summary
MHA and MQA both let query heads attend across token positions; MQA reduces cache size by producing one shared K/V head per token instead of a separate K/V head for every query head, while causal masking makes old decoder K/V safe to reuse during autoregressive generation.

Source alignment: Original class notes and the referenced Shazeer (2019) MQA paper, "Fast Transformer Decoding: One Write-Head is All You Need." Explanatory mini-examples and diagrams in this revision are pedagogical illustrations, not verbatim paper figures.